Wider Deep Learning Topics

Extended deep learning notes Sources: listed under References

What this file is really about:

$$\text{standard textbook and research topics} \rightarrow \text{the one idea each adds} \rightarrow \text{where to use it}$$

How to use this file. These topics are not on the course slides, but textbooks and interviews use them. Each section gives the central idea and its equation. Part B has checked code.

Step-up path

You need. The main deep learning sections.

1. Add a building block.
2. Choose an optimiser variant.
3. Certify a radius.
4. Weigh several tasks.
5. Derive generalised advantage.

You can stop when you can place each textbook topic in context.

Contents

Part A Maths From the Ground Up

A1 Building blocks

ResNeXt replaces a block's middle convolution with many parallel paths that share a topology, using grouped convolutions. **Squeeze-and-excitation** computes one weight per channel from global features, so channels can be emphasised. **ShuffleNet** uses cheap grouped convolutions and shuffles channels so that groups can exchange information.

Pixel shuffle upsamples by rearranging: Cr^2 channels at (H, W) become C channels at (rH, rW). It avoids the checkerboard pattern of transposed convolutions.

Instance norm normalises each image and each channel separately, removing per-image contrast, which helps style transfer.

DropConnect drops individual weights instead of units. **Stochastic depth** skips an entire residual block with a probability, and it rises with depth in a typical schedule. The test network uses every block, with its output scaled by the survival rate in the classic form.

A2 Optimisation methods

Sharpness-aware minimisation looks for flat minima by taking the gradient at a nearby worst-case point:

$$\epsilon = \rho \frac{\nabla L(w)}{\|\nabla L(w)\|}, \qquad w \leftarrow w - \eta \nabla L(w + \epsilon).$$

It costs two gradient computations per step. **Lookahead** keeps slow weights ϕ, and every k fast steps sets $\phi \leftarrow \phi + \alpha(w - \phi)$. **Warm restarts**, or SGDR, repeat cosine decay, often with each cycle longer than the last.

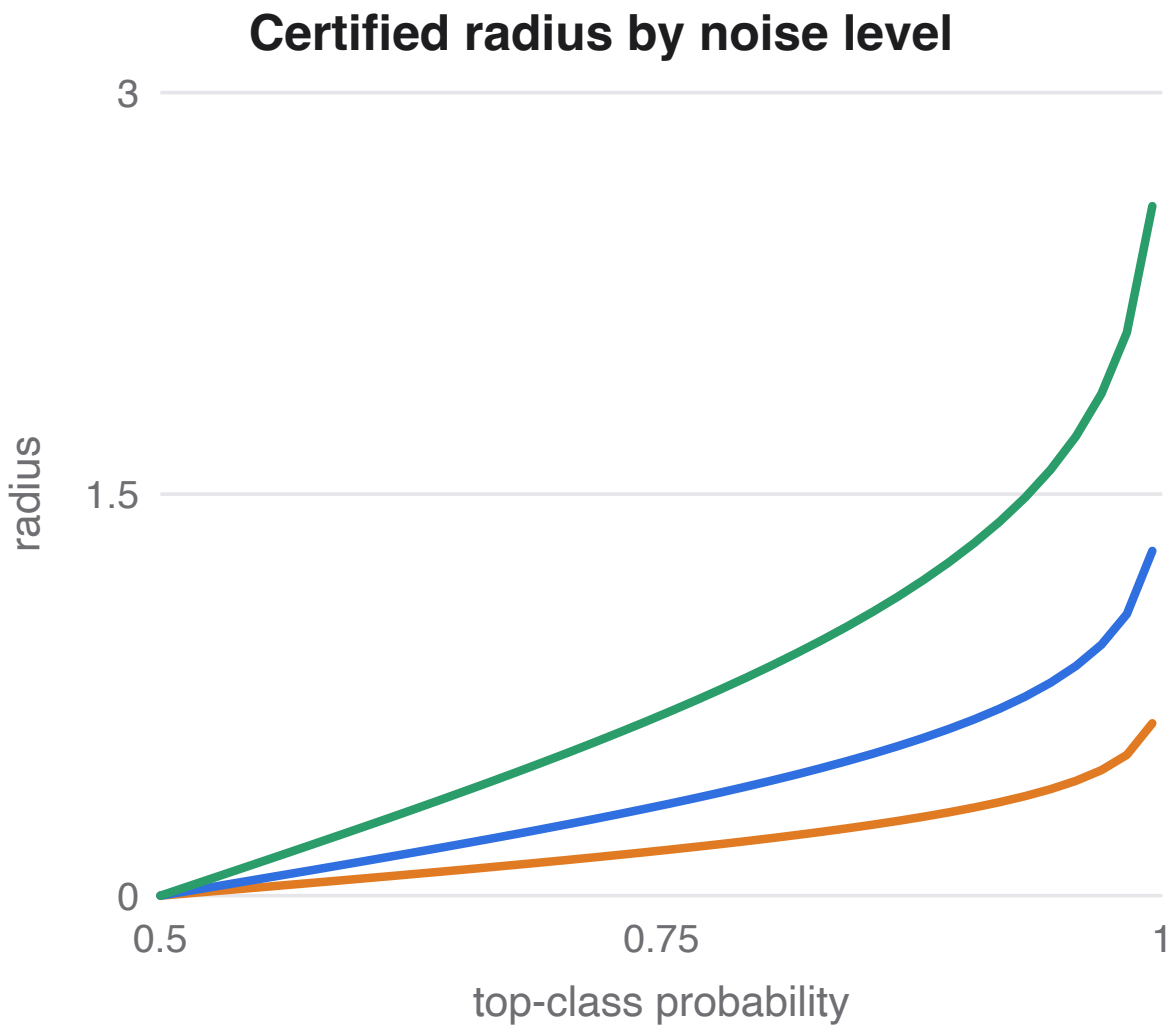
Orthogonal initialisation sets a matrix so that $W^\top W = I$. It preserves vector lengths exactly, which keeps gradients from growing or vanishing in deep linear chains.

A3 Designing and inspecting models

Neural architecture search automates design. Its parts are a search space, a search strategy (evolution, reinforcement learning or gradient methods such as DARTS) and a way to estimate quality cheaply, such

as weight sharing.

TCAV tests whether a human concept matters to a prediction. It trains a linear classifier between concept examples and random examples in a layer, takes the classifier's normal as the concept direction v, and measures how often $\nabla h(x) \cdot v > 0$ across a class.



A4 Robustness and privacy

Randomised smoothing turns a classifier f into $g(x) = \arg\max_c P(f(x+\varepsilon) = c)$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$. If the top class has probability at least $p_A > \tfrac{1}{2}$, then g is provably constant within an ℓ_2 radius

$$R = \sigma \, \Phi^{-1}(p_A).$$

Membership inference asks whether an example was in the training set. Loss on an example gives a simple attack, because models fit their own training points more closely. Better attacks compare against

shadow models, and differential privacy limits all of them.

A5 Learning across tasks and labels

Prototypical networks average each class's support embeddings to get a prototype c_k, and classify a query by softmax of $-\|f(x) - c_k\|^2$.

Domain-adversarial training (DANN) trains features that fool a domain classifier. A gradient reversal layer flips the sign of the gradient, so the features become domain-invariant.

Uncertainty weighting combines task losses with learned noise levels:

$$L = \sum_t \frac{1}{2\sigma_t^2} L_t + \log \sigma_t.$$

The optimum sets $\sigma_t^2 = L_t$, so noisy tasks receive less weight automatically.

Ordinal regression predicts ranked labels such as ratings. One threshold model is $P(y \le k \mid x) = \mathrm{sigmoid}(\theta_k - f(x))$, with increasing thresholds. It respects order, which plain classification ignores.

Weak supervision combines noisy cheap labels, such as rules or distant lookups, using a model of each source's accuracy. **Curriculum learning** orders examples from easy to hard.

A6 Transformer details

Attention pooling replaces averaging with a learned query that attends over a set, giving a weighted summary.

Rotary embeddings (RoPE) encode position by rotating query and key pairs of coordinates, $q_m = R(m\theta)q$. The dot product then depends only on the relative position:

$$\langle R(m\theta)q,\, R(n\theta)k \rangle = \langle R((m-n)\theta)q,\, k \rangle.$$

Sliding-window attention lets each token attend only to the last w tokens, so attention memory is bounded. **Sparse attention** methods reduce the $O(n^2)$ cost in other ways, such as atrous or block patterns.

A7 Reinforcement learning extras

Generalised advantage estimation mixes residuals $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$:

$$\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}.$$

Small λ has lower variance and more bias. $\lambda = 1$ is the full Monte Carlo return minus the baseline.

DDPG learns a deterministic actor for continuous control. **TD3** repairs its overestimation with two critics, delayed actor updates and smoothing noise on target actions.

AlphaZero runs tree search guided by a policy and value network, and trains on self-play. **MuZero** also learns the model it searches, predicting only reward, value and policy. **Decision Transformer** treats offline RL as sequence modelling of (return-to-go, state, action). **Diffusion policy** generates a short sequence of actions using diffusion, which represents several valid behaviours well.

A8 Geometry and sets

PointNet handles point clouds, which have no order. It applies the same network to each point and then a symmetric function such as max:

$$f(\{x_i\}) = \gamma\big(\max_i h(x_i)\big).$$

A9 Theory topics

A function is L**-Lipschitz** if $\|f(x) - f(y)\| \leq L\|x - y\|$. For a network, L is at most the product of layer spectral norms times activation slopes. Spectral normalisation limits each factor, which stabilises discriminators.

The **information bottleneck** seeks a representation that compresses the input while keeping label information:

$$\min_{p(t|x)} I(X;T) - \beta I(T;Y).$$

Deep equilibrium models define the output as a fixed point, $z^* = f(z^*, x)$, and differentiate through it with the implicit function theorem, using constant memory.

A10 Older and alternative models

A **Hopfield network** stores binary patterns with $W = \frac{1}{N}\sum_p \xi^p \xi^{p\top}$ (zero diagonal). Updates $s_i \leftarrow \operatorname{sign}(\sum_j W_{ij} s_j)$ only ever lower the energy $E = -\frac{1}{2} s^\top W s$, so a corrupted pattern can settle back. Capacity is about $0.14N$ random patterns.

A **restricted Boltzmann machine** has energy $E(v, h) = -a^\top v - b^\top h - v^\top W h$. There are no edges within a layer, so each layer can be sampled at once given the other. Training uses contrastive divergence.

Capsule networks replace scalar units with vectors whose length is a presence probability and whose orientation holds pose. **GFlowNets** sample discrete objects with probability proportional to a reward, which suits diverse molecule design. **Kolmogorov-Arnold networks** put learnable one-dimensional functions on edges, and sum them at nodes.

Part B In Code

B1 The smoothing radius

```
from scipy.stats import norm
for s, p in [(0.25, 0.9), (0.5, 0.9), (0.5, 0.99)]:
    print(s, p, round(s * norm.ppf(p), 3))
assert abs(0.5 * norm.ppf(0.9) - 0.6408) < 1e-3
```

B2 Sharpness-aware minimisation will not settle in a sharp minimum

```
import numpy as np
# A sharp basin at -2 with the lower loss, and a flat basin at 2.
L = lambda w: min(10 * (w + 2)**2 - 0.2, 0.3 * (w - 2)**2)
dL = lambda w, e=1e-5: (L(w + e) - L(w - e)) / (2 * e)
def train(rho, w):
    for _ in range(4000):
        g = dL(w); eps = rho * np.sign(g) if g else 0
        w = w - 0.01 * dL(w + eps)
    return w
print('from the sharp side: SGD', round(train(0.0, -1.75), 2), 'SAM', round(train(0.6, -1.7
print('from the flat side: SAM', round(train(0.6, 1.0), 2))
assert abs(train(0.0, -1.75) + 2) < 0.05 and abs(train(0.6, -1.75) + 2) > 0.3
assert abs(train(0.6, 1.0) - 2) < 0.1
```

B3 Hopfield recall

```
import numpy as np
rng = np.random.default_rng(0)
N = 200; P = 10
xi = rng.choice([-1, 1], (P, N))
W = xi.T @ xi / N; np.fill_diagonal(W, 0)
s = xi[0].copy(); flip = rng.choice(N, 40, replace=False); s[flip] *= -1
E = lambda v: -0.5 * v @ W @ v
before = E(s)
for _ in range(5):
    for i in rng.permutation(N):
        s[i] = 1 if W[i] @ s >= 0 else -1
overlap = (s == xi[0]).mean()
print('overlap', overlap, 'energy', round(before, 1), round(E(s), 1))
assert overlap > 0.98 and E(s) < before
```

B4 Generalised advantage from the recursion

```
import numpy as np
r = np.array([1.0, 0.0, 2.0, 1.0]); V = np.array([0.5, 0.4, 0.9, 0.3, 0.0]); g = 0.9
def gae(lam):
    A = np.zeros(4); a = 0
    for t in reversed(range(4)):
        d = r[t] + g * V[t+1] - V[t]; a = d + g * lam * a; A[t] = a
    return A
ret = [sum(g**(k-t) * r[k] for k in range(t, 4)) for t in range(4)]
assert np.allclose(gae(1.0), np.array(ret) - V[:4])
assert np.allclose(gae(0.0), r + g * V[1:] - V[:4])
print('GAE matches both ends:', gae(0.95).round(3))
```

B5 Rotary embeddings depend only on relative position

```
import numpy as np
R = lambda a: np.array([[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]])
q, k, th = np.array([0.3, 1.1]), np.array([-0.7, 0.4]), 0.21
a = (R(5 * th) @ q) @ (R(2 * th) @ k); b = (R(23 * th) @ q) @ (R(20 * th) @ k)
print(round(a, 9), round(b, 9)); assert abs(a - b) < 1e-12
```

B6 Uncertainty weighting and prototypes

```
import numpy as np, torch
for L_t in [0.5, 4.0]:
    s = np.linspace(0.05, 5, 200001); obj = L_t / (2 * s**2) + np.log(s)
    assert abs(s[obj.argmin()]**2 - L_t) < 1e-3
torch.manual_seed(0)
support = torch.tensor([[0., 0.], [0., 2.], [4., 0.], [4., 2.]]); y = torch.tensor([0, 0, 1
proto = torch.stack([support[y == c].mean(0) for c in [0, 1]])
query = torch.tensor([[0.5, 1.0]])
p = torch.softmax(-torch.cdist(query, proto)**2, 1)
print(proto.tolist(), p.round(decimals=3).tolist()); assert p[0, 0] > 0.99
```

Part C Practice

P1. With $\sigma = 0.5$ and $p_A = 0.5$, what radius does randomised smoothing certify?

$\Phi^{-1}(0.5) = 0$, so the radius is zero.

P2. How many channels must a pixel shuffle's input have for $C = 3$ and $r = 2$?

$Cr^2 = 12$.

P3. What does $\lambda = 0$ give in generalised advantage estimation?

The one-step temporal-difference residual.

P4. Why does a symmetric function let PointNet ignore order?

Max returns the same value for any permutation of its inputs.

P5. Two layers have spectral norms 3 and 2, with a 1-Lipschitz activation. Bound the network's Lipschitz constant.

$3 \times 1 \times 2 = 6$.

P6. Why does a Hopfield update never raise the energy?

Setting s_i to the sign of its local field makes $s_i h_i$ as large as possible, and with symmetric weights and no self-connections that term controls how the energy changes.